BEACON 학습 주석본

한 줄 요약

컨테이너가 실제로 쓰는 시스템콜과 capability만 허용하는 보안 정책을 자동으로 만든다. 기존 동적 분석은 한 가지 실행 환경만 관찰해서 놓치는 이벤트가 많았는데, BEACON은 Docker 옵션과 워크로드를 바꿔 가며 여러 환경을 흉내 내고 eBPF로 이벤트를 수집한 뒤, seccomp와 capability 정책으로 집행한다. 출처: Kang, Marin, You, Perino, Shin, Kim, Computers & Security 게재 승인본, arXiv 2512.00414. 유명성 교수님 공저, 교신저자 김진우 교수님.

읽는 순서

1) 초록과 1장 (정적 분석은 과대 추정, 동적 분석은 과소 추정이라는 문제 설정) 2) 2장 표 1 (환경에 따라 시스템콜이 달라지는 실제 예) 3) 3장 위협 모델 4) 4장은 그림 1(전체 구조)과 그림 2(eBPF 수집)를 먼저 보고 본문으로 5) 5장 평가는 5.2(11%, 22%), 5.4(오버헤드 5.6%), 5.7 표 5(CVE 차단율)만 먼저. 알고리즘 1과 식 (1)(2)는 두 번째 읽을 때 본다. 처음 읽을 때는 노란색만 따라가고, 두 번째에 초록 용어 메모를 읽는 것을 권한다.

표시 규칙

노랑: 핵심 주장, 결과 수치

초록: 전문용어. 오른쪽에 풀이

분홍: 한계, 가정, 위협 모델

①②… 빨간 번호 = 오른쪽 여백의 같은 번호 설명. → 표시 = EN683·한화비전과 연결되는 지점.

이 판에는 원문 쪽이 없다. 원문을 옆에 띄워 놓고 쪽 번호와 메모 번호로 따라 읽는다. 형광펜 문구는 원문에서 그대로 옮겨 적었다. 원문: https://arxiv.org/abs/2512.00414

용어집 (등장 순서)

이 논문의 집행 수단 두 가지. seccomp는 시스템콜을, capability는 root 권한 조각을 막는다. 둘 다 리눅스 커널 기본 기능이라 별도 설치가 필요 없다.

eBPF: 커널 안에서 안전하게 돌아가는 작은 프로그램. 커널을 다시 빌드하지 않고 이벤트를 관찰할 수 있다. 여기서는 수집만 하고 차단은 하지 않는다.

공격 표면 축소: 공격자가 건드릴 수 있는 입구(여기서는 커널 시스템콜과 권한)의 수를 줄이는 것.

동적 분석: 프로그램을 실제로 실행하면서 관찰하는 방법. 정적 분석은 실행하지 않고 코드나 바이너리만 읽는다.

VM과 컨테이너의 차이: VM은 게스트 커널을 따로 가진다. 컨테이너는 호스트 커널 하나를 모두가 공유한다. 그래서 커널 취약점 하나로 탈출이 가능하다.

최소 권한 원칙: 일에 꼭 필요한 권한만 준다. 이 논문 전체의 목표가 이 원칙을 자동으로 지키는 것이다.

NIST SP 800-190: 미국 표준기술연구소의 컨테이너 보안 지침. 커널 공격 표면을 줄이라고 권고한다.

system event: 이 논문에서는 시스템콜과 capability 요청을 합쳐 부르는 말.

실행 환경(environment): 컨테이너 행동을 바꾸는 외부 요인. 여기서는 Docker 실행 옵션과 들어오는 워크로드 두 가지.

corner case: 예외 처리처럼 평소에는 안 타는 경로. 여러 환경을 흉내 내야 이런 경로가 실행된다.

공개 코드: github.com/haney1357/BeaCon.

컨테이너 하드닝: 컨테이너가 가진 권한을 줄여 커널 공격 위험을 낮추는 작업 전반.

capability: root 권한을 약 40개 조각으로 나눈 것. 예: CAP_NET_RAW(raw 소켓), CAP_NET_BIND_SERVICE(1024 미만 포트). 필요한 조각만 준다.

환경 인식 프로파일링: 여러 실행 환경을 돌려 보며 관찰하는 것. 이 논문의 핵심 아이디어.

init 프로세스(tini): 컨테이너 안의 1번 프로세스. 끝난 자식 프로세스(좀비)를 정리하고 시그널을 전달한다.

1024 미만 포트(예: 80)에 바인드하는 권한. --network host일 때만 필요해진다.

cgroups: CPU·메모리 사용량 제한. namespace: 프로세스·네트워크·파일시스템을 컨테이너마다 따로 보이게 하는 격리. 둘이 컨테이너의 기본 뼈대다.

그림 1: Emulation Agent(환경 생성) → Monitoring Agent(eBPF 수집) → Decision Agent(정책 결정). 결과물은 seccomp 정책과 capability 목록.

seccomp-BPF: 시스템콜 번호(와 인자)를 BPF 필터로 검사해 허용·차단한다. Docker와 Kubernetes가 그대로 받아 쓰는 형식이다.

offline: 미리 오래 걸리는 프로파일링을 해 둔다. online: 컨테이너 실행 요청이 오면 저장된 정책을 바로 적용한다. → TDSnap과 같은 구조.

블랙박스 방식: 앱 내부 코드를 몰라도 되는 방식. 소스 코드를 받기 어려운 클라우드 환경에 맞다.

옵션 DB: 각 컨테이너에 쓸 수 있는 Docker 옵션과 그 옵션이 바꾸는 시스템 이벤트를 저장한다.

형식 문법: Docker 공식 문서는 사람이 읽는 글이라 옵션 값의 범위가 모호하다. 그래서 타입과 값 범위를 명확히 정의했다(표 2). → FuzzDocs의 문서 기반 입력 생성과 비슷한 문제의식.

워크로드 DB: DB 명령, REST API 요청처럼 컨테이너에 줄 입력 모음.

알고리즘 1: 옵션 값을 바꿔 가며 새 이벤트가 적게 나오면 보폭(step)을 키우고, 많이 나오면 줄인다. 가끔 무작위로 점프해 지역에 갇히지 않게 한다. 퍼징의 변이 전략과 비슷하다(추정).

휴리스틱: 모든 조합을 다 돌리면 시간이 폭발한다. 그래서 근사 규칙으로 줄인다.

무작위 변이: 이전 환경을 조금씩 바꿔 새 환경을 만든다.

tracepoint sys_enter: 커널에 미리 박혀 있는 고정 관찰점. 모든 시스템콜 진입 순간에 호출된다. 여기서 시스템콜 번호를 기록한다.

kprobe cap_capable: 커널 함수 아무 곳에나 거는 동적 관찰점. cap_capable은 커널이 capability를 검사할 때 부르는 함수라, 컨테이너가 어떤 권한을 요구했는지 알 수 있다.

eBPF 맵: 커널의 eBPF 프로그램과 사용자 공간 프로그램이 데이터를 주고받는 저장소. 키는 컨테이너 namespace, 값은 시스템콜·capability 비트맵.

unshare: 새 namespace를 만드는 시스템콜, 즉 컨테이너 시작 신호.
seccomp/prctl/capset: 컨테이너에 보안 정책이 걸리는 순간. 그 이후 이벤트만 센다.

Decision Agent: 모인 이벤트 중 무엇을 허용할지 점수로 정한다.

CVSS: 취약점 심각도 점수 0~10. CVE: 공개 취약점 고유 번호. 허용한 이벤트에 연결된 CVE 중 가장 높은 CVSS로 보안 점수를 낮춘다.

기능 점수: 여러 환경 중 이 정책으로 정상 동작하는 환경의 비율. 1이면 모든 환경에서 안 죽는다.

평가 질문 Q1~Q6: 추가 이벤트 발견, 탐색 공간 축소, 점수 조절, 오버헤드, 정적 분석 비교, 실제 공격 차단.

BCC: Python에서 eBPF C 코드를 컴파일·로드하게 해 주는 도구 모음. → 임베디드 보드에서는 컴파일러와 커널 헤더가 필요해 그대로 쓰기 어렵다(추정). EN683 커널의 eBPF 지원 여부는 확인하지 못했다.

YCSB: Yahoo가 만든 DB 부하 생성 벤치마크. 읽기·쓰기·스캔 비율을 정해 요청을 보낸다.

표 3: 워크로드 W1~W8. 연산 종류, 필드 수·길이, 스레드 수를 바꿔 서로 다른 커널 경로를 건드리게 한다.

5.2절: 환경 흉내가 실제로 이벤트를 더 찾는지 검증.

5.4절: eBPF 수집만 켰을 때의 비용. ring buffer는 커널에서 사용자 공간으로 이벤트를 넘기는 순환 버퍼.

Spec 1~3: 보안 점수 목표를 0 → 0.5 → 0.7로 올리고 기능 목표를 1 → 0.5 → 0.25로 내린 세 가지 설정.

비교 대상: Chestnut, Sysfilter, Confine 모두 정적 분석 도구. 소스나 바이너리를 읽어 시스템콜을 추정한다.

CDF(누적분포함수): x값 이하인 비율. 곡선이 왼쪽으로 치우치면 대부분 이미지의 값이 작다는 뜻.

완화율: 특정 CVE 공격에 필요한 시스템콜이나 capability가 정책에서 빠져 있으면 그 컨테이너는 보호된 것으로 센다.

예: CVE-2017-7308은 raw 소켓 경로 취약점. Confine은 0/71, BEACON은 70/71을 막는다. capability(NET_RAW)를 다뤘기 때문.

Dirty CoW: copy-on-write 처리의 경쟁 조건(race condition)으로 읽기 전용 파일을 덮어쓰는 리눅스 커널 취약점(2016).

CVE-2020-14386: raw 소켓 패킷 처리의 메모리 손상 버그. socket·setsockopt로 커널을 패닉시킨다.

LSM: 커널의 보안 정책 확장 틀. AppArmor, SELinux가 대표적이다. 파일 접근 같은 자원 단위 강제 접근 통제(MAC)를 한다. → 임베디드 리눅스 하드닝에서도 쓰는 계층.

표 6: 기존 도구 비교. BEACON만 동적 분석이면서 capability, 환경 인식, 정책 조절을 모두 지원한다고 주장한다.

Speaker: 컨테이너를 시작 단계와 실행 단계로 나눠 시스템콜을 프로파일링. Timeloops: 시스템콜 정책을 학습하는 방식.

Confine: 컨테이너 이미지의 바이너리와 라이브러리를 분석해 쓸 수 있는 시스템콜을 추정하는 정적 분석 도구. 이 논문의 주 비교 대상.

하이브리드: 정적 분석으로 후보를 뽑고 동적 분석으로 검증. 소스 코드가 필요해서 클라우드에서는 쓰기 어렵다고 본다.

원문 1쪽

Linux Seccomp and Capabilities
① 이 논문의 집행 수단 두 가지. seccomp는 시스템콜을, capability는 root 권한 조각을 막는다. 둘 다 리눅스 커널 기본 기능이라 별도 설치가 필요 없다.

eBPF-based monitor
② **eBPF:** 커널 안에서 안전하게 돌아가는 작은 프로그램. 커널을 다시 빌드하지 않고 이벤트를 관찰할 수 있다. 여기서는 수집만 하고 차단은 하지 않는다.

low-overhead policies
③ **하이라이트 3:** 실제 Docker 이미지로 평가했고, 기존 방법보다 커널 공격을 더 많이 막는다.

원문 2쪽

Attack surface reduction
① **공격 표면 축소:** 공격자가 건드릴 수 있는 입구(여기서는 커널 시스템콜과 권한)의 수를 줄이는 것.

system events
② **system event:** 이 논문에서는 시스템콜과 capability 요청을 합쳐 부르는 말.

(VMs)
③ **VM과 컨테이너의 차이:** VM은 게스트 커널을 따로 가진다. 컨테이너는 호스트 커널 하나를 모두가 공유한다. 그래서 커널 취약점 하나로 탈출이 가능하다.

principle of least privilege
④ **최소 권한 원칙:** 일에 꼭 필요한 권한만 준다. 이 논문 전체의 목표가 이 원칙을 자동으로 지키는 것이다.

dynamic analysis to simulate
⑤ **동적 분석:** 프로그램을 실제로 실행하면서 관찰하는 방법. 정적 분석은 실행하지 않고 코드나 바이너리만 읽는다.

16.5%
⑥ **초록 수치:** 상위 15개 컨테이너에서 환경을 바꿔 시스템콜을 평균 16.5% 더 찾는다. 본문(1장)은 옵션 11%, 워크로드 22%로 나눠 적는다.

45 known vulnerabilities
⑦ **보안 평가 규모:** 알려진 커널 CVE 45개로 차단 효과를 측정. 실제 공격 재현(PoC)은 2개.

over 300 syscalls
⑧ **문제의 크기:** Docker 기본값은 300개 넘는 시스템콜 중 44개만 막고, capability 38개 중 14개를 준다.

NIST container security guidelines
⑨ **NIST SP 800-190:** 미국 표준기술연구소의 컨테이너 보안 지침. 커널 공격 표면을 줄이라고 권고한다.

원문 3쪽

underestimation
① **동적 분석의 한계:** 실행해 보지 않은 경로의 시스템콜은 못 본다. 정책에서 빠지면 나중에 실제 운영에서 컨테이너가 죽는다.

estimation of required privileges
② **정적 분석의 한계:** 호출될 수도 있는 것을 전부 넣는다. 안전하지만 정책이 넓어져 공격 표면이 크다.

ignore Linux capabilities
③ 기존 연구 대부분이 capability를 다루지 않았다. BEACON의 차별점 중 하나.

execution environments
④ **실행 환경(environment):** 컨테이너 행동을 바꾸는 외부 요인. 여기서는 Docker 실행 옵션과 들어오는 워크로드 두 가지.

exception
⑤ **corner case:** 예외 처리처럼 평소에는 안 타는 경로. 여러 환경을 흉내 내야 이런 경로가 실행된다.

11% and 22%
⑥ **결과 1:** Docker 옵션을 고려하면 시스템콜 11%, 워크로드를 고려하면 22%를 더 찾는다. 이만큼을 기존 동적 분석은 놓친다.

87%
⑦ **결과 2:** 알려진 취약점에서 평균 87% 컨테이너를 보호, 정적 분석은 60%. 단 5.7절 표 5의 평균은 85.9%로 적혀 있다.

Linux Container Hardening.
⑧ **컨테이너 하드닝:** 컨테이너가 가진 권한을 줄여 커널 공격 위험을 낮추는 작업 전반.

https://github.com/
⑨ **공개 코드:** github.com/haney1357/BeaCon.

원문 4쪽

rt_sigtimedwait
① **표 1:** --init 옵션 하나만 붙여도 필요한 시스템콜이 2개 늘어난다. 한 환경에서 만든 정책은 다른 환경에서 컨테이너를 죽일 수 있다는 근거.

Need for Restricting Both Syscalls and Capabilities.
② **capability:** root 권한을 약 40개 조각으로 나눈 것. 예: CAP_NET_RAW(raw 소켓), CAP_NET_BIND_SERVICE(1024 미만 포트). 필요한 조각만 준다.

CAP_NET_RAW
③ **핵심 예:** socket 시스템콜을 막으면 정상 앱이 깨진다. 대신 CAP_NET_RAW만 빼면 raw 소켓만 막힌다. 시스템콜과 capability를 함께 봐야 하는 이유.

Need for Environment-Aware
④ **환경 인식 프로파일링:** 여러 실행 환경을 돌려 보며 관찰하는 것. 이 논문의 핵심 아이디어.

reap zombies and forward signals
⑤ **init 프로세스(tini):** 컨테이너 안의 1번 프로세스. 끝난 자식 프로세스(좀비)를 정리하고 시그널을 전달한다.

Operation not permitted.
⑥ seccomp가 막으면 이렇게 실패한다. 정책이 너무 좁으면 정상 동작도 이 오류로 끝난다.

CAP_NET_BIND_SERVICE
⑦ **1024 미만 포트(예:** 80)에 바인드하는 권한. --network host일 때만 필요해진다.

Threat Model.
⑧ **위협 모델:** 공격자가 이미 컨테이너 안에 들어왔다고 가정한다. 목표는 커널 취약점으로 탈출해 호스트나 옆 컨테이너를 장악하는 것.

원문 5쪽

the broader the container-kernel interface
① **핵심 논리:** 허용된 시스템콜이 많을수록 커널 취약점을 찾을 확률이 커진다. 그래서 줄인다.

out root privileges
② **가정 1:** 컨테이너는 root 없이 돌고 cgroups·namespace로 격리되어 있다.

(e.g., cgroups [32] and namespaces [33])
③ **cgroups:** CPU·메모리 사용량 제한. namespace: 프로세스·네트워크·파일시스템을 컨테이너마다 따로 보이게 하는 격리. 둘이 컨테이너의 기본 뼈대다.

container images are not compromised
④ **가정 2:** 내려받은 이미지는 깨끗하다. 프로파일링 중에는 공격이 없다고 본다. 이 가정이 깨지면 악성 행동이 정책에 들어간다.

Emulating Agent
⑤ **그림 1:** Emulation Agent(환경 생성) → **Monitoring Agent(eBPF 수집) → Decision Agent(정책 결정).** 결과물은 seccomp 정책과 capability 목록.

seccomp-BPF and Linux capabilities
⑥ **seccomp-BPF:** 시스템콜 번호(와 인자)를 BPF 필터로 검사해 허용·차단한다. Docker와 Kubernetes가 그대로 받아 쓰는 형식이다.

Online and
⑦ **offline:** 미리 오래 걸리는 프로파일링을 해 둔다. online: 컨테이너 실행 요청이 오면 저장된 정책을 바로 적용한다. **→ TDSnap과 같은 구조.**

원문 6쪽

black-
① **블랙박스 방식:** 앱 내부 코드를 몰라도 되는 방식. 소스 코드를 받기 어려운 클라우드 환경에 맞다.

Container Runtime Options Database.
② **옵션 DB:** 각 컨테이너에 쓸 수 있는 Docker 옵션과 그 옵션이 바꾸는 시스템 이벤트를 저장한다.

Formal Syntax
③ **형식 문법:** Docker 공식 문서는 사람이 읽는 글이라 옵션 값의 범위가 모호하다. 그래서 타입과 값 범위를 명확히 정의했다(표 2). **→ FuzzDocs의 문서 기반 입력 생성과 비슷한 문제의식.**

Workloads Database.
④ **워크로드 DB:** DB 명령, REST API 요청처럼 컨테이너에 줄 입력 모음.

원문 7쪽

Algorithm 1: Option value mutation
① **알고리즘 1:** 옵션 값을 바꿔 가며 새 이벤트가 적게 나오면 보폭(step)을 키우고, 많이 나오면 줄인다. 가끔 무작위로 점프해 지역에 갇히지 않게 한다. 퍼징의 변이 전략과 비슷하다(추정).

4.2. Environment Emulation Heuristics
② **휴리스틱:** 모든 조합을 다 돌리면 시간이 폭발한다. 그래서 근사 규칙으로 줄인다.

Randomized Testing Environment Mutation.
③ **무작위 변이:** 이전 환경을 조금씩 바꿔 새 환경을 만든다.

Optimization for Reducing Emulation Space.
④ **핵심 최적화:** 옵션 A와 B를 함께 쓴 결과 ≈ A 결과 ∪ B 결과로 보고, 조합 대신 하나씩만 돌린다. 5.3절에서 검증.

원문 8쪽

Hash Map
① **eBPF 맵:** 커널의 eBPF 프로그램과 사용자 공간 프로그램이 데이터를 주고받는 저장소. 키는 컨테이너 namespace, 값은 시스템콜·capability 비트맵.

TRACEPOINT
② **tracepoint sys_enter:** 커널에 미리 박혀 있는 고정 관찰점. 모든 시스템콜 진입 순간에 호출된다. 여기서 시스템콜 번호를 기록한다.

kprobe
③ **kprobe cap_capable:** 커널 함수 아무 곳에나 거는 동적 관찰점. cap_capable은 커널이 capability를 검사할 때 부르는 함수라, 컨테이너가 어떤 권한을 요구했는지 알 수 있다.

4.3. Container Event Monitoring
④ **중요한 구분:** eBPF는 관찰(수집)만 한다. 실제 차단은 나중에 seccomp와 capability로 한다.

Tracepoint probes
⑤ **unshare:** 새 namespace를 만드는 시스템콜, 즉 컨테이너 시작 신호. seccomp/prctl/capset: 컨테이너에 보안 정책이 걸리는 순간. 그 이후 이벤트만 센다.

4.4. Container Policy Decision
⑥ **Decision Agent:** 모인 이벤트 중 무엇을 허용할지 점수로 정한다.

Security and Functionality Scores.
⑦ 두 점수로 보안과 기능의 균형을 사용자가 조절한다. 하나만 극단으로 올리면 다른 쪽이 나빠진다.

Common Vul-
⑧ **CVSS:** 취약점 심각도 점수 0~10. CVE: 공개 취약점 고유 번호. 허용한 이벤트에 연결된 CVE 중 가장 높은 CVSS로 보안 점수를 낮춘다.

원문 9쪽

The functionality score
① **기능 점수:** 여러 환경 중 이 정책으로 정상 동작하는 환경의 비율. 1이면 모든 환경에서 안 죽는다.

Policy Generation.
② **정책 생성:** CVSS가 임계값을 넘는 이벤트는 빼고, 기능 점수 목표를 만족하는지 확인한다.

5. Evaluation
③ **평가 질문 Q1~Q6:** 추가 이벤트 발견, 탐색 공간 축소, 점수 조절, 오버헤드, 정적 분석 비교, 실제 공격 차단.

1320 lines of Python code
④ **구현 규모:** Python 1320줄 + eBPF C 170줄(BCC 툴체인). 작아서 직접 읽어 볼 만하다.

BPF Compiler Collection (BCC)
⑤ **BCC:** Python에서 eBPF C 코드를 컴파일·로드하게 해 주는 도구 모음. **→ 임베디드 보드에서는 컴파일러와 커널 헤더가 필요해 그대로 쓰기 어렵다(추정).** EN683 커널의 eBPF 지원 여부는 확인하지 못했다.

178 Docker container images
⑥ **데이터셋:** Docker Hub 공식 이미지 178개.

YCSB benchmark framework
⑦ **YCSB:** Yahoo가 만든 DB 부하 생성 벤치마크. 읽기·쓰기·스캔 비율을 정해 요청을 보낸다.

원문 10쪽

Table 3
① **표 3:** 워크로드 W1~W8. 연산 종류, 필드 수·길이, 스레드 수를 바꿔 서로 다른 커널 경로를 건드리게 한다.

Scope and Generality.
② **범위 한계:** 평가는 DB·웹 서버 같은 서버형 이미지 중심이다. 다른 유형은 입력을 따로 만들어야 한다.

Figure 3:
③ **그림 3:** Docker 옵션을 적용하면 이미지마다 시스템콜 수가 늘어난다(진한 막대가 BEACON).

94 images expose network ports

④ 167개 중 94개가 기본으로 포트를 연다. 서버형 이미지가 다수라서 YCSB 평가가 대표성이 있다는 주장.

원문 11쪽

Figure 4:
① **그림 4:** 워크로드를 넣으면 시스템콜이 늘어난다. 평균 22%.

5.2. Effectiveness of Environment Emulation
② **5.2절:** 환경 흉내가 실제로 이벤트를 더 찾는지 검증.

51% (85 images)
③ 167개 중 51%가 옵션 적용 시 시스템콜 5% 이상 증가, 6%는 capability가 20% 넘게 증가.

39% of containers (28 images)
④ 워크로드 적용 시 71개 중 39%가 시스템콜 5% 이상 증가.

62.5% of the 384
⑤ **5.3절 결과:** 62.5%는 A∪B와 A+B 동시 결과가 완전히 같고, 나머지도 시스템콜 1개 차이. 조합을 다 안 돌려도 된다는 근거.

5.4. Performance of Event Monitoring
⑥ **5.4절:** eBPF 수집만 켰을 때의 비용. ring buffer는 커널에서 사용자 공간으로 이벤트를 넘기는 순환 버퍼.

average of 5.6%
⑦ **오버헤드:** 처리량 평균 5.6% 감소, CPU 0.8~9% 증가. 단, 이 비용은 오프라인 프로파일링 때만 든다. 실제 운영 때는 seccomp 필터만 남는다.

원문 12쪽

Spec. 1
① **Spec 1~3:** 보안 점수 목표를 0 → 0.5 → 0.7로 올리고 기능 목표를 1 → 0.5 → 0.25로 내린 세 가지 설정.

Figure 6:
② **그림 6:** 모니터링을 켜도 처리량 차이가 작다.

Figure 7:
③ **그림 7:** CPU 사용량 상대 증가 0.8~9%.

policy size decreases monotonically
④ 보안 목표를 높일수록 허용 시스템콜 수가 꾸준히 준다. 점수로 정책 크기를 조절할 수 있다는 결과.

Chestnut [15], Sysfil-
⑤ **비교 대상:** Chestnut, Sysfilter, Confine 모두 정적 분석 도구. 소스나 바이너리를 읽어 시스템콜을 추정한다.

원문 13쪽

Figure 8:
① **그림 8:** CDF 곡선이 왼쪽일수록 허용 시스템콜이 적다. BEACON이 가장 왼쪽.

cumulative distribution function (CDF)
② **CDF(누적분포함수):** x값 이하인 비율. 곡선이 왼쪽으로 치우치면 대부분 이미지의 값이 작다는 뜻.

5.7. Mitigation of Known Vulnerabilities
③ **완화율:** 특정 CVE 공격에 필요한 시스템콜이나 capability가 정책에서 빠져 있으면 그 컨테이너는 보호된 것으로 센다.

headless
④ **평가 범위 한계:** 사람 손 없이 뜨는 이미지 71개만 썼다. 수동 설정이나 오케스트레이터가 필요한 이미지는 빠졌다.

CVE-2017-7308
⑤ **예:** CVE-2017-7308은 raw 소켓 경로 취약점. Confine은 0/71, BEACON은 70/71을 막는다. capability(NET_RAW)를 다뤘기 때문.

Average Mitigation Ratio
⑥ **표 5 평균:** Confine 60.5%, BEACON 보안 우선 85.9%, 기능 우선 60.47%. 초록의 87%와 약간 다르다.

원문 14쪽

(CVE-2016-5195) Dirty CoW
① **Dirty CoW:** copy-on-write 처리의 경쟁 조건(race condition)으로 읽기 전용 파일을 덮어쓰는 리눅스 커널 취약점(2016).

following six syscalls
② 공격에 필요한 시스템콜 6개 중 madvise만 드물다. 71개 중 22개 컨테이너는 어떤 환경에서도 madvise를 안 쓰므로 BEACON이 막는다.

(CVE-2020-14386) Raw-socket
③ **CVE-2020-14386:** raw 소켓 패킷 처리의 메모리 손상 버그. socket·setsockopt로 커널을 패닉시킨다.

Kernel Panic Attack.
④ **공격 결과:** 커널 패닉은 호스트 전체가 멈추는 것. 컨테이너 하나가 모든 고객 서비스를 내릴 수 있다.

원문 15쪽

capability-based policy
① **결론:** 시스템콜 정책(Confine, Speaker)으로는 못 막고, CAP_NET_RAW를 뺀 capability 정책으로 막는다.

Dependency to Workloads.
② **한계 1:** 결과가 운영자가 준 워크로드의 품질에 달려 있다. 동적 분석 공통의 약점.

Linux Security Modules (LSMs)
③ **LSM:** 커널의 보안 정책 확장 틀. AppArmor, SELinux가 대표적이다. 파일 접근 같은 자원 단위 강제 접근 통제(MAC)를 한다. → **임베디드 리눅스 하드닝에서도 쓰는 계층.**

Including Syscall Parameters in Policy Generation.
④ **한계 2:** 시스템콜 인자까지는 거르지 않는다. 인자 필터링은 향후 과제.

Handling Zero-Day Vulnerabilities.
⑤ **한계 3:** CVSS가 없는 제로데이는 보안 점수에 반영되지 않는다.

원문 16쪽

Table 6
① **표 6:** 기존 도구 비교. BEACON만 동적 분석이면서 capability, 환경 인식, 정책 조절을 모두 지원한다고 주장한다.

Dynamic Analysis-based Solutions.
② **Speaker:** 컨테이너를 시작 단계와 실행 단계로 나눠 시스템콜을 프로파일링. Timeloops: 시스템콜 정책을 학습하는 방식.

Confine [14] is the state-of-the-art
③ **Confine:** 컨테이너 이미지의 바이너리와 라이브러리를 분석해 쓸 수 있는 시스템콜을 추정하는 정적 분석 도구. 이 논문의 주 비교 대상.

원문 17쪽

Hybrid Solutions.
① **하이브리드:** 정적 분석으로 후보를 뽑고 동적 분석으로 검증. 소스 코드가 필요해서 클라우드에서는 쓰기 어렵다고 본다.

8. Conclusion
② **결론:** 여러 환경을 흉내 낸 동적 분석으로 시스템콜과 capability 정책을 함께 만든다. → **카메라 펌웨어의 서비스별 seccomp 정책 생성으로 옮겨 볼 수 있다(추정).** EN683의 seccomp 지원은 확인하지 못했다.